AI-BASED HUMAN ACTIVITY RECOGNITION PLATFORM

Comprehensive Technical Specification, Architecture & Team Role Breakdown
Execution Mode: 100% Localhost • **Hardware Target:** NVIDIA GeForce RTX 3050 (6GB VRAM)

---

1. Executive Summary & Problem Statement

Project Mission: To develop a local, privacy-first, web-accessible deep learning platform that allows non-technical users to ingest custom action datasets, automatically inspect and validate video streams using OpenCV, train genuine 3D Convolutional Neural Networks (R3D-18) using hardware-accelerated GPU resources, and run automated, real-time activity predictions on newly captured footage.

The Core Problem: Photos vs. Motion (Why 2D Computer Vision Fails)

Standard computer vision models are built for 2D static images (Height × Width). However, human activities (such as walking, running, falling, punching, or stretching) are fundamentally defined by *temporal movement across time*. A static photo of a raised arm cannot determine whether an individual is waving goodbye, stretching, or throwing a punch. 3D Spatiotemporal modeling processes sequential frames across time simultaneously to extract velocity, trajectory, cadence, and limb displacement.

Limitations in Existing Industry Solutions:

- **Cloud & Privacy Dilemma:** Commercial platforms (AWS Rekognition, Google Cloud Vertex AI) require uploading sensitive surveillance video over the internet, violating HIPAA medical privacy and enterprise data security protocols.
- **Complexity & Tooling Barrier:** Academic research repositories (MMAction2, PyTorchVideo) require terminal proficiency, manual format conversion scripts, and complex Docker configs beyond the reach of healthcare or security staff.
- **Hardware Overload:** Most video transformers demand massive enterprise GPUs (A100/H100 with 40–80GB VRAM) and immediately crash with Out-Of-Memory (OOM) errors on consumer hardware.

2. System Architecture & Technical Stack

The application utilizes a completely decoupled Client-Server architecture running locally without external web service dependencies:

Layer	Technology	Architectural Justification
Frontend UI	React 18 + TypeScript + Vite	Modern reactive Single-Page Application (SPA) with strict type safety, modular routing, and sub-second Hot Module Replacement.
Styling & UX	Tailwind CSS + Lucide Icons	Responsive dark-theme cyberpunk interface, accessibility badges, upload modals, and telemetry gauges.

Live Charting	Recharts Library	Dynamic real-time loss and accuracy curves updated via polling without requiring full browser reloads.
Backend REST API	FastAPI (Python 3.13) + Uvicorn	Asynchronous ASGI server hosted on port 8001 providing high-concurrency request handling and auto-generated Swagger UI.
Database & ORM	SQLite + SQLAlchemy 2.0	Zero-config persistent single-file database (`har_platform.db`) with dual async (`aiosqlite`) and synchronous engines.
Video Processing	OpenCV (`cv2`)	Direct frame decoding, container integrity inspection, aspect-ratio resizing, and SHA-256 duplicate hashing.
Deep Learning	PyTorch 2.14 + Torchvision	R3D-18 backbone, Kinetics-400 pretrained transfer weights, autograd backpropagation, and Cross-Entropy loss.
GPU Acceleration	NVIDIA CUDA 12.6 + AMP	Targeted for RTX 3050 (6GB). PyTorch Automatic Mixed Precision (FP16) cuts VRAM usage by 50% and doubles throughput.
Async Worker	Python `threading` + Queue	Background daemon thread decoupled from FastAPI's event loop to prevent web server freezes during long multi-hour training runs.

3. Team Roles & Work Distribution (3 Members)

To ensure balanced contributions, the project was cleanly divided into three distinct architectural layers: Frontend/UI, Backend/Data Pipeline, and Machine Learning/GPU Acceleration.

Member 1: Frontend Engineer & UI/UX Lead

Primary Ownership: *Web Interface, Real-Time Charting, Client-Side Routing & UX Validation*

• **Architecture & Tooling:** Built the React 18 SPA in TypeScript with Vite bundler, styling all screens with Tailwind CSS.
• **5 Core Pages Developed:**
1. *Dashboard:* Real-time GPU telemetry card (RTX 3050 status, memory) and global summary counters.
2. *Dataset Management:* Drag-and-drop ZIP uploaders, multi-file video selectors, and class distribution charts.
3. *Training Center:* Visual hyperparameter configuration forms (epochs, batch size, learning rates, train/val/test sliders).
4. *Model Registry:* Visual catalog of saved `.pth` checkpoints with benchmark metrics and class mappings.
5. *Analysis Module:* Video upload panel with top action label, confidence meter, and millisecond latency timers.
• **Real-Time Visualizations:** Integrated Recharts for live rendering of Loss vs. Epoch and Accuracy vs. Epoch trajectories.
• **Client Error Handling:** Designed Axios interceptors to intercept backend validation errors and format them into readable alerts.

Member 2: Backend & System Pipeline Engineer

Primary Ownership: *REST API Engine, SQLite Database Modeling & OpenCV Video Validation Engine*

• **FastAPI REST Architecture:** Developed all API routes (`/datasets`, `/training`, `/models`, `/analysis`, `/system`) on port 8001.
• **Database Schema Design:** Built relational models in SQLAlchemy 2.0 with cascading delete relationships (`ON DELETE CASCADE`) to prevent foreign-key orphaned records when datasets are removed.
• **OpenCV Validation Pipeline (`video_service.py`):**
- Opens every incoming video with `cv2.VideoCapture` to verify codec decodability and flag corrupt containers.
- Extracts video properties: spatial resolution, total frame count, FPS, and duration.
- Implemented **SHA-256 Cryptographic Hashing** to detect identical duplicate clips and prevent test-set data leakage.
• **Ingestion Security:** Implemented safe archive extraction protecting against Zip-Bomb exploits, scaling capacity up to 20 GB and 100,000 files for massive datasets like Real-Life Violence.

Member 3: Machine Learning & Computer Vision Lead

Primary Ownership: *3D CNN Modeling (R3D-18), Spatiotemporal Tensors, GPU Optimization & Training Worker*

• **Spatiotemporal Deep Learning Model:** Implemented **R3D-18 (3D ResNet-18)** in PyTorch. Unlike 2D CNNs, R3D-18 employs 3D convolution kernels ($K_t × K_h × K_w$) sliding across space and consecutive time frames simultaneously.
• **Transfer Learning:** Leveraged Kinetics-400 pretrained weights, severing the final linear head and attaching custom classifier layers for user-specific activity categories.
• **Tensor Construction:** Engineered sampling transforms extracting uniform 16-frame sequences resized to 112×112, producing standardized 5D PyTorch tensors shaped `[Batch, 3, 16, 112, 112]`.
• **Dual-Format Support:** Extended loader to seamlessly ingest both raw video clips (`.mp4`, `.avi`) and sequential image-frame directories (`.jpg`, `.png`).
• **NVIDIA RTX 3050 (6GB) Optimization:** Implemented **PyTorch Automatic Mixed Precision (AMP / FP16)**, reducing VRAM footprint by 50% to prevent CUDA Out-of-Memory crashes.
• **Background Training Thread:** Built a multi-threaded daemon worker (`training_worker.py`) using thread-safe queues and SQLite sync sessions, ensuring training never freezes the FastAPI web server.

4. Deep Learning Methodology & Mathematical Formulations

1. 2D vs. 3D Convolutional Mathematics:
In standard 2D Convolution, feature values at position (x, y) are calculated over spatial kernel dimensions P and Q:

$$v_{ij}^{xy} = f\left(\sum_m \sum_p \sum_q w_{ijm}^{pq} v_{(i-1)m}^{(x+p)(y+q)} + b_{ij} \right)$$

In our **3D ResNet (R3D-18)**, an additional temporal dimension *r* across kernel temporal length R is introduced:

$$v_{ij}^{xyz} = f\left(\sum_m \sum_p \sum_q \sum_r w_{ijm}^{pqr} v_{(i-1)m}^{(x+p)(y+q)(z+r)} + b_{ij} \right)$$

This formulation allows the network to mathematically capture feature displacement and velocity vectors across consecutive video frames.

2. ResNet Residual Skip Connections:
To prevent the vanishing gradient problem in deep networks, residual blocks compute:

$$y = F(x, \{W_i\}) + x$$

where *x* and *y* are the input and output vectors, and *F* is the residual mapping to be learned. The identity shortcut ensures error gradients flow directly back to early layers without degrading.

3. Loss Function & Softmax Classification:
Model parameters are optimized by minimizing **Categorical Cross-Entropy Loss**:

$$L_{CE} = -\sum_{c=1}^{C} y_c \log(\blacksquare_c)$$

where raw logits z_i are converted to probability distributions via the Softmax function:

$$\blacksquare_i = \exp(z_i) / \sum_{j=1}^{C} \exp(z_j)$$

5. Real-World Applications & Future Scope

Domain	Problem Solved	Deployment Impact
Healthcare & Elderly Care	Sudden accidental patient falls in unmonitored rooms.	Instant alert to nursing stations within 100ms; 100% on-premise privacy with zero cloud video transmission.
Public Safety & Security	Operator fatigue monitoring dozens of live CCTV cameras.	Automated violence, assault, and fight detection flagging security feeds with high-confidence alarms.
Industrial Safety	OSHA violations: workers entering hazardous machinery zones or unsafe ladder climbing.	Automated machine shut-off triggers and compliance incident logging.
Sports & Physical Therapy	Manual exercise form evaluation and repetition counting.	Algorithmic athletic technique analysis and improper form warnings.

Planned Future Roadmap:

1. **Live RTSP CCTV Streaming:** Implementing circular rolling frame buffers to analyze live IP camera feeds 24/7 without manual video uploads.
2. **Multi-Person Tracking:** Coupling YOLOv8 object detection and MediaPipe Pose estimation to classify multiple individuals simultaneously in a single frame.
3. **Edge AI Quantization:** Converting PyTorch `.pth` models to INT8 TensorRT engines for deployment onto $200 NVIDIA Jetson micro-computers embedded directly inside camera enclosures.

6. Project Verification & Readiness Summary

Metric / Feature	Status	Verification Detail
Frontend Codebase	PASSED	Zero TypeScript errors; production bundle built via Vite.
Backend API Routing	PASSED	FastAPI operational on port 8001; full Swagger docs available.
OpenCV Verification	PASSED	Corrupt file handling, FPS, duration, and SHA-256 duplicate checks verified.
GPU Acceleration	PASSED	NVIDIA GeForce RTX 3050 (6GB) recognized via CUDA 12.6 + FP16 AMP.
Data Ingestion	PASSED	Scaled to 20 GB / 100,000 files; supports both video and image sequences.
Execution Model	100% LOCAL	Zero cloud dependency; complete data confidentiality guaranteed.